

Sliding Window Complete DSA Notes (Beginner to Advanced)

1. What is Sliding Window?

Sliding Window is an optimization technique used on arrays and strings.

Instead of recalculating a range again and again, maintain a moving window.

Usually converts:

$O(n^2)$

into

$O(n)$

Used for:

- Subarrays
- Substrings
- Maximum/minimum ranges
- Continuous segments

2. Basic Idea

Maintain two pointers:

left

right

Window:

[left right]

Expand using right pointer.

Shrink using left pointer.

3. Types of Sliding Window

Two major types:

1. Fixed Size Window

Window length is constant.

Example:

Maximum sum subarray of size K.

2. Variable Size Window

Window grows and shrinks.

Example:

Longest substring without repeating characters.

4. Fixed Window Template

Steps:

1. Add new element.
2. When size reaches k:
calculate answer.
3. Remove left element.
4. Move window.

5. Variable Window Template

Steps:

1. Expand right.
2. Update window state.
3. While condition invalid:
shrink left.
4. Update answer.

Python Fixed Window Example

```
# Maximum sum subarray size k
def maxSum(nums,k):
    window=0
    ans=0

    for right in range(len(nums)):
        window+=nums[right]

        if right>=k-1:
            ans=max(ans,window)

            window-=nums[right-k+1]

    return ans
```

C++ Fixed Window Example

```
int maxSum(vector<int>& nums,int k){
    int sum=0;
    int ans=0;

    for(int r=0;r<nums.size();r++){
        sum+=nums[r];

        if(r>=k-1){
            ans=max(ans,sum);

            sum-=nums[r-k+1];
        }
    }

    return ans;
}
```

Java Fixed Window Example

```
int maxSum(int[] nums,int k){
    int sum=0;
    int ans=0;

    for(int r=0;r<nums.length;r++){
```

```

sum+=nums[r];

if(r>=k-1){
ans=Math.max(ans, sum);

sum-=nums[r-k+1];
}
}

return ans;
}

```

Python Variable Window Template

```

def slidingWindow(nums):
    left=0

    for right in range(len(nums)):

        # add nums[right]

        while invalid_condition:

            # remove nums[left]

            left+=1

        # update answer

```

6. Longest Substring Without Repeating Characters

Classic variable sliding window.

LC3

Maintain:

HashSet

If duplicate appears:

remove from left.

Python LC3 Example

```

def lengthOfLongestSubstring(s):
    seen=set()
    left=0
    ans=0

    for right in range(len(s)):

        while s[right] in seen:
            seen.remove(s[left])
            left+=1

        seen.add(s[right])

        ans=max(
            ans,
            right-left+1
        )

    return ans

```

7. Sliding Window + HashMap

Used when frequency matters.

Examples:

- Anagrams
- Minimum Window Substring
- Character Replacement

8. Sliding Window Maximum

Advanced variation.

Use:

Deque

Maintain decreasing order.

Front always stores maximum.

Complexity:

$O(n)$

9. Common Patterns

Fixed Window:

- Maximum sum size K
- Average size K
- First negative number

Variable Window:

- Longest substring
- Minimum window
- At most K distinct

Advanced:

- Monotonic Queue
- Prefix + Window

Important Leetcode Problems

LC3:

Longest Substring Without Repeating Characters

LC76:

Minimum Window Substring

LC209:

Minimum Size Subarray Sum

LC239:

Sliding Window Maximum

LC424:

Longest Repeating Character Replacement

LC567:

Permutation in String

LC438:
Find All Anagrams

Complexity Cheatsheet

Brute Force:

$O(n^2)$

Sliding Window:

$O(n)$

Space:

$O(1)$

or

$O(\text{characters/hashmap size})$